

2026 FALL · WEEK 5

Advanced Server Programming

The data underneath · Tables, keys, SQL · H2 inside your project

Prof. Youngsil Lee | Department of Computer Science
Dongseo University — International College

Today — from a List to a table

1 Assignment 2, and three rules

What I saw in your repositories, and what is marked from today.

2 Tables, keys, relationships

Room and Reservation as tables. A fast SQL review — we review, we do not re-teach.

3 H2 inside your project

Add a real database to the Room API, open its console, and break its rules on purpose.

4 Let Spring build the tables

schema.sql and data.sql at startup, and the gap we close in Week 6.

Restart your server and every room you created is gone. Today we build the place where data survives that.

ASSIGNMENT 2

What I saw in your repositories

Personal feedback for each of you is on eClass. These four patterns came up more than once.

1 It did not run

Missing gradlew or settings.gradle, .java files uploaded to the top of the repository, a JDK 25/26 toolchain on Java 21 machines.

2 HTTP inside the service

RoomService threw ResponseStatusException. Status codes belong to the controller; the service throws IllegalArgumentException.

3 A class, not the interface

The service asked for InMemoryRoomRepository. In Week 6 that class is replaced — ask for RoomRepository.

4 README ≠ API

The README said PUT on a missing room gives 404; the code created the room and answered 200.

Nobody built the Reservation extension. Good — it is today's design exercise, and we start it on paper.

FROM TODAY

Three rules for every assignment

1

Clone it into a new folder and run your README

That is exactly what I do. If a file is not committed, the project does not run, and you lose points.

2

Late work counts, with 1 point off

I grade by your commit times. Work committed after the deadline is graded with a 1-point deduction.

3

Java 21 in build.gradle

You may use another JDK at home, but the toolchain line must say 21 so the lab machines can build it.

```
git clone https://github.com/<you>/<repo>.git test-run && cd test-run && ./gradlew bootRun
```

PART 1

The data underneath

Why a database, and how Room and Reservation become tables

THE PROBLEM

Restart the server, lose everything

Where do our rooms actually live today?

```
@Repository
public class InMemoryRoomRepository
    implements RoomRepository {

    private final List<Room> rooms =
        new ArrayList<>(List.of(...));
}
```

1

POST /api/rooms

201 · Location /api/rooms/4

2

Stop and restart bootRun

a new JVM, a new List

3

GET /api/rooms/4

404 Not Found

A List lives exactly as long as the JVM. Real data has to live somewhere that outlives any one run of your server.

What a database does that a List cannot

1

Survives restarts

Data is written to disk, not held in one program's memory.

2

One truth for many servers

Two copies of your app can share the same rooms. Two Lists cannot.

3

Answers any question

SQL can ask things you did not write a Java method for.

4

Guards its own rules

Keys and constraints reject bad data even when your code forgets to check.

Card 4 matters most today. You wrote the capacity rule in the service last week. Today the database gets a copy of it.

REVIEW

One table, all the words

The room table after data.sql has run.

ID (PK)	NAME	CAPACITY
1	Seminar A	8
2	Study Pod	4
3	Rooftop Room	12

Table one kind of thing — rooms

Row one room

Column one fact about every room

Primary key the column that tells rows apart

Foreign key a column that points at another table's primary key

The id is the same idea as our Room.id — but now the database, not an AtomicLong, hands out the next number.

The same Room, written twice

JAVA

```
public record Room(  
    Long id,  
    String name,  
    int capacity) { }
```

SQL

```
CREATE TABLE room (  
    id          BIGINT AUTO_INCREMENT PRIMARY KEY,  
    name        VARCHAR(100) NOT NULL,  
    capacity    INT          NOT NULL,  
    CONSTRAINT chk_room_capacity  
        CHECK (capacity BETWEEN 1 AND 20)  
);
```

JAVA

SQL

Long

BIGINT

String

VARCHAR(n) — you choose n

int

INT

NEW HERE

NOT NULL and CHECK. The record accepts anything; the table refuses a nameless room or a capacity of 0.

A room has many reservations

On paper, in Excel, in Paint — any way you like. Fill in three sample rows for each table.

1

What columns does a reservation need?

Who, which room, from when, until when. What types?

2

Where does the link live?

Does room point at reservation, or reservation at room?

3

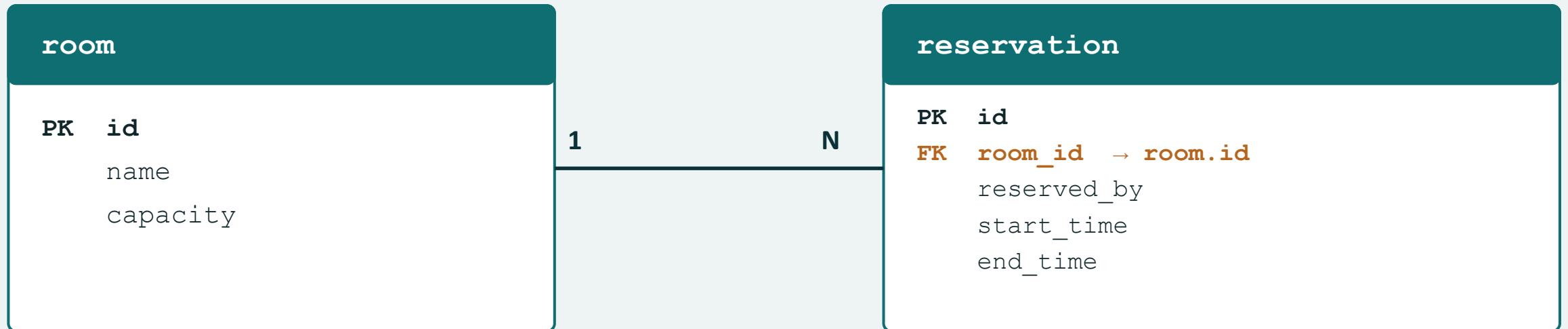
Why not a list inside room?

Could room have a column `reservation_ids = "1,2,4"`? What goes wrong?

Hint: a cell holds one value. Try writing Seminar A's three reservations — where do they fit? Keep your design: it goes into Assignment 3's README.

ANSWER

The foreign key lives on the "many" side



WHY NOT A LIST IN ROOM?

One cell, one value. "1,2,4" cannot be joined, checked by a key, or cleaned up when reservation 2 is deleted.

IN JAVA TERMS

Each Reservation knows its room. The room does not keep a list — you ask the database for "reservations where room_id = 1".

Two families of statements, plus the rules

DDL — shape of the data

```
CREATE TABLE  
ALTER TABLE  
DROP TABLE
```

DML — the data itself

```
INSERT INTO ... VALUES  
SELECT ... FROM ... WHERE  
UPDATE ... SET ... WHERE  
DELETE FROM ... WHERE
```

Constraints — the rules

```
PRIMARY KEY  
NOT NULL  
FOREIGN KEY ... REFERENCES  
CHECK (...)
```

UPDATE and DELETE without WHERE change every row. In the console there is no undo. Read the WHERE before you press Run.

How a SELECT is actually evaluated

You write it top to bottom. The database runs it in a different order.

```
SELECT    rm.name, COUNT(r.id) AS n
FROM      room rm
LEFT JOIN reservation r
          ON r.room_id = rm.id
WHERE     rm.capacity >= 4
GROUP BY  rm.id, rm.name
HAVING    COUNT(r.id) > 0
ORDER BY  n DESC;
```

- 1 FROM / JOIN — build the rows
- 2 WHERE — drop rows
- 3 GROUP BY — make groups
- 4 HAVING — drop groups
- 5 SELECT — pick columns, name aliases
- 6 ORDER BY — sort (aliases work here)

That is why WHERE n > 0 fails: the alias n does not exist yet when WHERE runs. Use HAVING.

INNER JOIN drops the room nobody booked

Reservations per room, written two ways.

`JOIN + COUNT(*)`

NAME	RESERVATIONS
Seminar A	3
Study Pod	2

`LEFT JOIN + COUNT(r.id)`

NAME	RESERVATIONS
Seminar A	3
Study Pod	2
Rooftop Room	0

COUNT(*) counts rows; COUNT(r.id) counts reservations. After a LEFT JOIN, Rooftop Room still has one row — with r.id = NULL. COUNT(*) would say 1.

PART 2

H2 inside your project

A real database with no installation, running inside your Spring Boot app

A database that lives inside your app

1 Embedded

It is one dependency in build.gradle. Nothing to install on the lab machine.

2 In memory (today)

jdbc:h2:mem:roomdb lives as long as the app runs — like the List, but with SQL.

3 A web console

/h2-console in your browser: type SQL, see tables, no extra tools.

4 MODE=MySQL

H2 accepts MySQL-style SQL, so what you write today runs on MySQL next week.

Today H2, next week MySQL. H2 for learning and tests; MySQL is what a deployed server talks to. Same SQL, different URL.

Choose your base project

Weeks 5 and 6 need the Week 4 layers. Check your own code first.

A

Your service asks for RoomRepository

...and the interface has an InMemoryRoomRepository implementing it.

Keep working in your own repository. Pull, check it runs, and go on.

B

It does not, or it does not run

Download week5-starter.zip from eClass. Copy its contents into your repository and commit:
"Start from the Week 5 starter".

The starter is the Week 4 reference solution. Controller → Service → RoomRepository, constructor injection, Swagger UI, api.http 1–14.

STEP 1

Three lines in build.gradle

Add them to dependencies. Then reload Gradle and restart the app — both, every time.

```
dependencies {  
    // keep the lines you already have, then add:  
  
    implementation 'org.springframework.boot:spring-boot-starter-jdbc'  
    implementation 'org.springframework.boot:spring-boot-h2console'  
    runtimeOnly 'com.h2database:h2'  
}
```

starter-jdbc

a DataSource, and running schema.sql at startup

h2console

the /h2-console page (a separate module since Spring Boot 4)

h2

the database itself

Did it work? The startup log must say H2 console available at '/h2-console'. No line = reload Gradle (the elephant icon in IntelliJ) and restart.

STEP 2

Tell Spring where the database is

src/main/resources/application.properties

```
spring.application.name=room-api

spring.datasource.url=jdbc:h2:mem:roomdb;MODE=MySQL;DATABASE_TO_LOWER=TRUE
spring.datasource.username=sa
spring.datasource.password=

spring.h2.console.enabled=true
```

mem:roomdb

an in-memory database with a fixed name —
you type this name in the console

MODE=MySQL

accept MySQL syntax; lower-case table names
like MySQL

console.enabled

turn the web console on (development only)

STEP 3

Restart, then open the console

<http://localhost:8080/h2-console>

Driver Class

`org.h2.Driver`

JDBC URL

`jdbc:h2:mem:roomdb`

User Name

`sa`

Password

`(empty)`

IF IT FAILS

404 on /h2-console → Gradle was not reloaded, or the app was not restarted. Check the log line from Step 1.

Error on Connect → type only
`jdbc:h2:mem:roomdb`. The database already exists with its settings; the part after ; is not needed here.

Same name, same database. Your app already opened roomdb in MySQL mode. The console joins it by name — and it is empty until Lab A.

Write schema.sql yourself

CREATE TABLE statements in a file named schema.sql, then run them in the console. Use these exact names.

room

id	BIGINT, generated by the database, primary key
name	text up to 100 characters, required
capacity	whole number, required, only 1 to 20

reservation

id	BIGINT, generated by the database, primary key
room_id	required, must be the id of an existing room
reserved_by	text up to 50 characters, required
start_time	date and time, required
end_time	date and time, required

Order matters: create room before reservation. A foreign key cannot point at a table that does not exist yet.

Write data.sql yourself

One INSERT per row. Insert the rooms in this order so the database gives them ids 1, 2, 3.

ROOM	
NAME	CAPACITY
Seminar A	8
Study Pod	4
Rooftop Room	12

CHECK YOURSELF

SELECT * FROM room; → 3 rows

SELECT * FROM reservation; → 5 rows

RESERVATION			
ROOM_ID	RESERVED_BY	START_TIME	END_TIME
1	Mina	2026-10-06 10:00:00	2026-10-06 11:00:00
1	Omar	2026-10-06 13:00:00	2026-10-06 15:00:00
2	Mina	2026-10-06 09:00:00	2026-10-06 10:00:00
1	Lucas	2026-10-07 10:00:00	2026-10-07 12:00:00
2	Aiko	2026-10-07 14:00:00	2026-10-07 15:00:00

Predict first, then run each one

Write your prediction down. Then read what the database says.

1

```
INSERT INTO reservation (room_id, reserved_by, start_time, end_time)
VALUES (99, 'Ghost', '2026-10-06 10:00:00', '2026-10-06 11:00:00');
```

room 99 does not exist

2

```
INSERT INTO room (name, capacity) VALUES ('Closet', 0);
```

capacity 0

3

```
DELETE FROM room WHERE id = 1;
```

room 1 has reservations

All three are refused by the database, not by Java. Your service checks capacity; the table checks it again. That is defense in depth.

Eight questions for the data

Q1–Q6 in class, Q7–Q8 at home. Save all eight in queries.sql under their questions — it is part of Assignment 3.

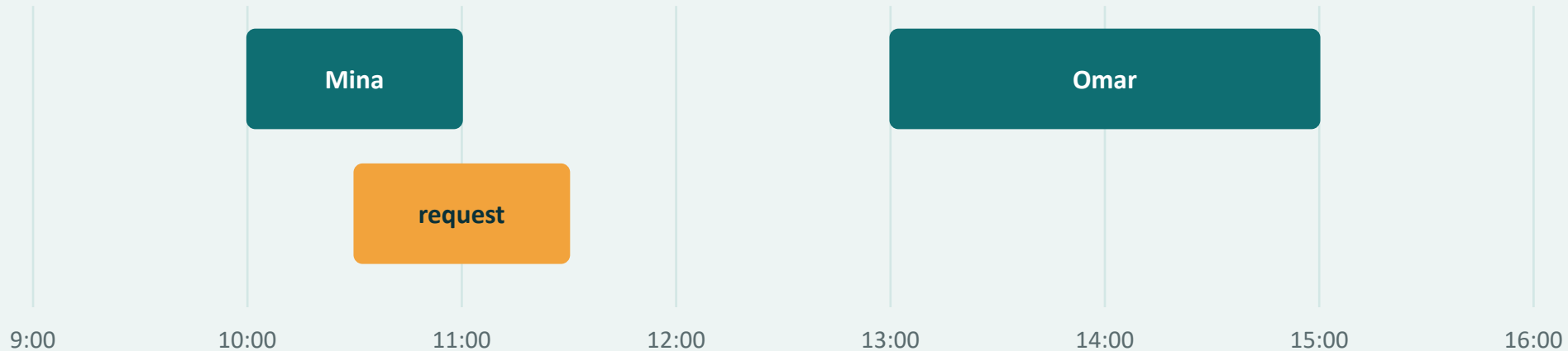
Q1	Rooms for 6 or more people, largest first	2 rows	Q5	Reservations per room, with JOIN	2 rows
Q2	Every reservation made by Mina	2 rows	Q6	Same as Q5, but every room — 0 included	3 rows
Q3	Every reservation with its room's name	5 rows	Q7	Rooms that have never been reserved	1 row
Q4	Seminar A's reservations on 6 October 2026	2 rows	Q8	Rooms with more than two reservations	1 row

Row counts let you check yourself. A different count means a different answer — look again at the JOIN or the WHERE.

CHALLENGE

Is room 1 free from 10:30 to 11:30 on 6 October?

Find every reservation that overlaps the requested time.



```
WHERE room_id = 1
      AND start_time < '2026-10-06 11:30:00' -- starts before it ends
      AND end_time   > '2026-10-06 10:30:00' -- ends after it starts
```

WHY THIS MATTERS

In Week 7 this query becomes a rule: the service refuses a booking that overlaps.

PART 3

Let Spring build the tables

schema.sql and data.sql at startup — and the gap Week 6 closes

Two files, run every time the app starts

```
src/main/resources/  
├─ application.properties  
├─ schema.sql    ← CREATE TABLE ...  
└─ data.sql      ← INSERT INTO ...
```

DO IT NOW

Move the schema.sql and data.sql you wrote in Lab A into this folder. Restart. Open the console: the tables are already there.

- 1 DataSource connects to jdbc:h2:mem:roomdb
- 2 schema.sql runs — tables exist
- 3 data.sql runs — 3 rooms, 5 reservations
- 4 Your beans are created
- 5 Tomcat starts on port 8080

For an in-memory database Spring runs them by default. Nothing to add to application.properties. (For MySQL next week, that changes.)

TRY IT

Restart once more — what survives?

1

In the console

```
INSERT INTO room (name, capacity) VALUES ('Library', 10);
```

2

Restart bootRun

the JVM ends, the in-memory database ends with it

3

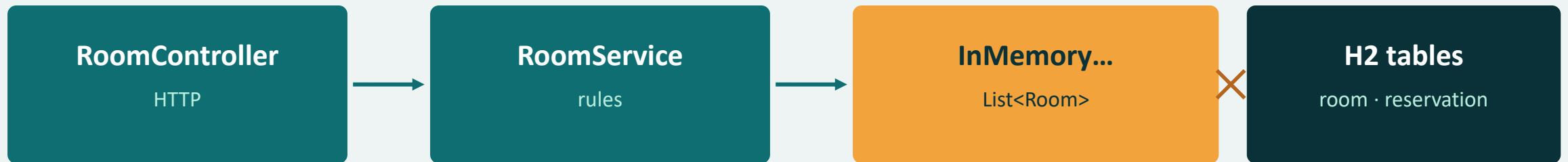
SELECT * FROM room;

3 rows — Library is gone; data.sql put the 3 rooms back

mem means the memory of this JVM. H2 gave us tables, keys and SQL — not survival. MySQL runs as its own program, so next week your rows outlive a restart.

Two worlds that do not know each other

Run `api.http 7` (POST a room), then `SELECT * FROM room` in the console.



WHAT YOU SEE

The API says 201 and `GET /api/rooms` shows 4 rooms. The console still shows 3. The API never touched the database.

WHY

RoomService talks to RoomRepository, and the only RoomRepository we have keeps a List. The tables are right there, unused.

Week 6 replaces exactly one box.

Two swaps, and the service does not change

1 H2 → MySQL

We start class by changing the URL, user and password in `application.properties`. `schema.sql` and your queries stay the same — that is why we used `MODE=MySQL` today.

2 InMemoryRoomRepository → JPA

A JPA repository takes the `List`'s place behind the `RoomRepository` interface. `RoomService` and `RoomController` stay exactly as they are.

Swap 1 needs MySQL on your machine before class. Next slide.

Install MySQL on the machine you use in class

Lab computer or laptop — whichever you will sit at next week.

1 Already have MySQL 8.0 or newer?

Do not reinstall. Just check that you can log in.

2 No MySQL yet

Install MySQL Community Server 8.4 LTS (Windows installer or macOS DMG).

3 Remember the root password

Write it down. Next week it goes into application.properties.

4 Prove it works (optional)

Log in, run `SELECT VERSION();` and upload a screenshot to eClass.

```
mysql -u root -p          then          SELECT VERSION();
```

ASSIGNMENT 3

The Room API, with a database underneath

Same repository. Due before Week 6 class (Monday 23:59). Submit the GitHub link on eClass.

H2	The three dependencies and application.properties from today. The app starts and /h2-console opens.	HOW IT IS MARKED 1. I clone it and follow your README. It must build and run. 2. The tables and data exist at startup. 3. Each query gives the right rows. Late work: 1 point off.
Startup SQL	schema.sql and data.sql in src/main/resources create both tables and the sample data.	
queries.sql	Q1–Q8 and the challenge, at the top of the repository, each under its question as a comment.	
README	A Database section: the two tables and their relationship, and how to open the console.	
MySQL	(Optional) Your SELECT VERSION(); screenshot, uploaded to eClass with the link.	

Commit after each step. "Add H2 and schema.sql" — not "update".

NEXT WEEK

Week 6 — the repository gets a real database

MySQL first, then JPA.

1

Swap the database

Same schema.sql, same queries — now on MySQL, and your rows survive a restart.

2

Entities and JPA

Room becomes an entity. A JPA repository replaces InMemoryRoomRepository.

3

Nothing else moves

RoomService and RoomController do not change. Week 4's interface pays off.

Before next week: MySQL installed and logged in once. Without it, you will spend Week 6 installing.

QUESTIONS

Three checks before you leave

1. Which table holds the foreign key, and why that one?
2. Why did Rooftop Room disappear in Q5 but not in Q6?
3. After a restart, why was Library gone while the three rooms came back?

Commit and push before you log out of the lab machine.